Python によるサウンドプログラミング

Psy Trance 風電子音楽の生成

1. プロジェクト概要

本プロジェクトでは、Python を用いた電子音楽の生成に取り組んだ。NumPy による波形生成と配列操作を中心に、Kick、Bass、Lead の 3 つの音響要素をプログラム上で合成し、Psy Trance 風のリズムとサウンドを構築した。
一般的な DAW による音楽制作とは異なり、音の周波数、振幅、時間的変化をコードで直接制御することで、デジタル音響合成の基本的な仕組みを理解することを目的とした。

2. 制作背景とコンセプト

本プロジェクトの着想は、Live Coding による音楽制作への関心から生まれた。
Live Coding は、プログラミング言語を用いて音楽の構造やパラメータを操作する表現手法であり、電子音楽や実験音楽の分野で活用されている。特に、コードによってリズムやメロディを制御する点に魅力を感じ、Python を用いた音楽制作を試みた。
音楽ジャンルには、普段から親しんでいる Psy Trance を選択した。Psy Trance は、規則的な Kick、反復的な Rolling Bass、幻想的な Lead などを特徴とする。これらの要素は配列操作やループ処理と相性が良く、プログラミングによる音響合成の学習に適していると考えた。

3. 開発環境と使用技術

開発言語・ライブラリ

Python

NumPy：波形生成、配列演算、音声信号の処理

sounddevice：生成した音声信号の再生

主な技術

正弦波・鋸歯状波による波形合成

振幅エンベロープによる音量制御

BPM に基づくリズム生成

for ループによる音符とメロディの生成

Kick に連動した Bass の音量制御

複数音声信号のミキシングと正規化

4. システム設計

制作にあたって、まず Logic Pro を用いて楽曲全体の構造を検討し、その後 Python による音響生成の基本的なフレームワークを構築した。
BPM、サンプリングレート、1 小節あたりのステップ数を設定し、NumPy を用いて時間軸と波形データを生成した。
音響構成は、以下の 3 つの要素に分けて実装した。

4.1 Kick の生成

Kick には正弦波を使用し、時間とともに周波数と振幅が変化することで、打楽器らしい低音を表現した。
各拍に独立した振幅エンベロープを適用し、4 つ打ちのリズムを構成した。
開発初期には、配列の切り取り位置やエンベロープの適用方法に問題があり、各拍の Kick が正しく再生されなかった。
そこで、以下の処理に修正した。
kick[i:i+length] += wave * env
これにより、各拍に生成した波形を適切な位置へ加算できるようになった。

4.2 Rolling Bass の生成

Bass には鋸歯状波を使用し、Psy Trance 特有の反復的な低音パターンを生成した。
BPM に同期した Gate 処理によって音量を周期的に変化させ、Rolling Bass のリズム感を表現した。
また、Kick と Bass の低音域が重なりすぎないよう、Kick の波形に基づく簡易的な Sidechain 風の音量制御を導入した。
sidechain = 1 - np.clip(kick * 8, 0, 1)
この処理では、Kick の信号に応じて Bass の振幅を調整している。ただし、一般的なコンプレッサーによる Sidechain 処理とは異なる簡易的な実装である。

4.3 Lead の生成

Lead は、メロディを配列として定義し、for ループによって各音符の波形を生成する方法を採用した。
音符ごとに減衰エンベロープを適用することで、音量が時間とともに自然に減少するようにした。
また、以下の処理によってメロディを繰り返した。
step % len(melody)
これにより、限られた音符の組み合わせから反復的な旋律パターンを構成した。

5. ミキシングと最終実装

生成した Kick、Bass、Lead をそれぞれ適切な音量バランスに調整し、加算することで最終的な音声信号を作成した。
ミキシング時には、配列の長さが一致しないことによる ValueError が発生したため、以下の処理を用いて長さを統一した。
length = min(len(kick), len(bass), len(lead))
さらに、合成後の信号を最大振幅に基づいて正規化し、過大な振幅によるクリッピングを抑制した。
最終的に、4 つ打ちの Kick、Rolling Bass、反復する Lead を組み合わせた Psy Trance 風の基本的なサウンドパターンを生成することができた。

6. 制作上の課題と学び

本プロジェクトでは、特に以下の点が課題となった。

音符の長さと配列インデックスの対応

各拍におけるエンベロープの適用

Kick と Bass の音量バランス

異なる長さの音声配列の統合

複数の波形を加算した際の振幅管理

これらの問題を修正する過程で、音声信号を配列として扱う方法や、サンプリングレートと時間軸の関係について理解を深めることができた。
DAW では視覚的・直感的に操作していた音楽的要素を、数値や数式、プログラムによって表現することで、音響合成の仕組みをより具体的に理解できた。

7. 今後の展望

今後は、以下の機能を追加し、より柔軟な音楽生成システムへ発展させたい。

LFO による周期的な音色変化

フィルター処理による周波数特性の制御

音色やリズムパターンのバリエーション追加

音声生成処理の関数化とコード構造の改善

パラメータを変更しながら音楽を生成する Live Coding 的な操作への発展

将来的には、音響合成だけでなく、アルゴリズムによる音楽生成やインタラクティブな音楽システムへの応用も検討したい。

8. 参考資料

OXXO：Python による音声生成

Python 音轨与音声合成全攻略

Sound from a NumPy Array in Python

Introduction to Sound Synthesis with Python

Code a Wavetable Synth with Python in 6 Minutes

Qiita：Python 音声合成に関する参考記事